Which Part of the Loop Helps? A Preregistered Decomposition of Recursive Self-Improvement for Vision-Language-Action Policies

Anonymous CVPR submission

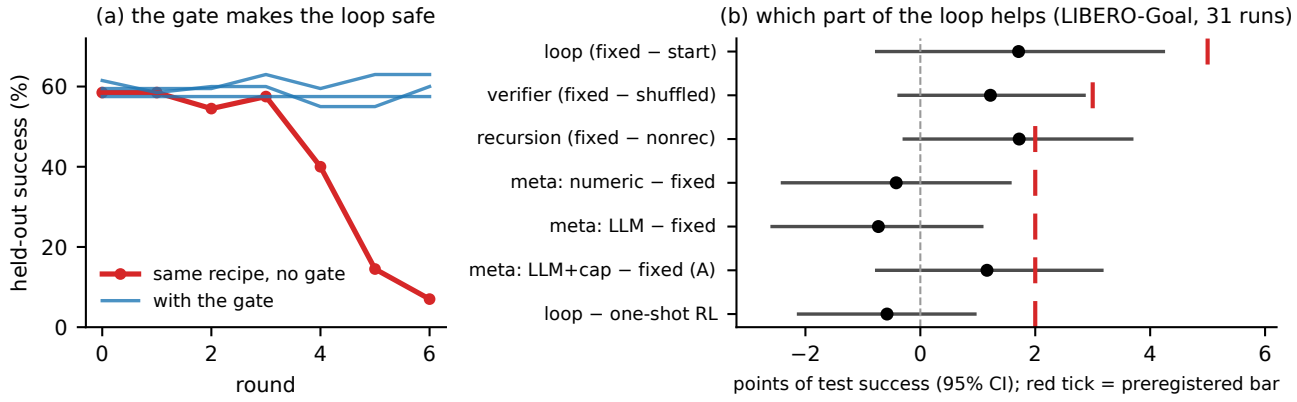


Figure 1. **What a preregistered decomposition of recursive self-improvement shows on a VLA policy.** (a) The verifier gate is what makes self-training safe: the same behaviour-cloning recipe collapses from 58.5% to 5.5% when every new version is accepted, and never falls below its start when each version must beat its parent on fresh paired placements. (b) Taking the loop apart one switch at a time (31 LIBERO-Goal runs, sealed official test, replicate-averaged): every ingredient's point estimate is positive except the meta level's, and none reaches its preregistered bar (red ticks). Recursion and the verifier are the largest contributors; an LLM that edits the training procedure adds nothing once the procedure works.

Abstract

Recursive self-improvement (RSI) promises a policy that practises on tasks it writes for itself, keeps only what an external verifier confirms, and revises its own training procedure. We ask whether this promise holds for vision-language-action (VLA) policies, and whether any gain is due to the recursive and meta-level parts of the loop rather than to plain reinforcement learning. We build a framework in which every ingredient is a switch: a verifier gate that accepts or rejects each new version on fresh held-out placements, recursion (practice generated by the latest policy or frozen at the initial one), a verifier (real or permuted success labels), a meta level (none, numeric search, or an LLM that diagnoses round records and edits the training procedure inside a declared space), and a compute-matched one-shot RL baseline. All claims were preregistered with sealed official tests, public amendments, and replicate-averaged reads. On LIBERO-Goal with OpenVLA-OFT (31 runs, four seeds, two GPU platforms) the gated loop never degrades a policy and gains $+1$ to $+2$ points; recursion contributes $+1.7$ points (95% CI $[-0.3, +3.7]$) and the verifier $+1.2$ $[-0.4, +2.9]$, but neither reaches its preregistered bar, and no meta-level variant beats a fixed, hand-tuned procedure (all within ± 1.2 points; the same null on a second, separately registered study). We show why: the object-level loop reaches the same plateau under every procedure in the space, so there is nothing for a meta level to fix, even though the LLM's diagnoses are correct. The meta level only becomes distinguishable from RL when the starting procedure fails; a preregistered self-repair study (sealed, to be read on 2026-11-08) tests exactly that. In a second domain (code world models, four LLMs), the same loop gives verified-feedback gains of $+2.6$ to $+7$ points and recursion is significant at eight rounds ($+2.8$ $[+1.2, +4.5]$ on CRUXEval-O). Finally, we measure that a published VLA-RL checkpoint trained on the benchmark's official initial states is inflated by $+16.8$ points $[+9.8, +23.4]$ relative to fresh placements. We release the framework, every preregistration and all run

records.

1. Introduction

A learning system is *recursively self-improving* (RSI) when the thing being improved is also the thing doing the improving: the current model writes its own practice problems, attempts them, keeps the attempts an external verifier confirms, trains on them, and the improved model writes the next round's problems. Taken one step further, the system also revises the *procedure* by which it improves. Recent language-model results in this direction are striking [2, 5, 23–25], reinforcement learning with verifiable rewards (RLVR) has become the standard recipe for reasoning models [3, 20], and robot learning has started to follow: a vision-language-action (VLA) policy can be improved by RL in simulation from a sparse success signal alone [13, 15].

Two questions are usually left open in such reports. First, *which part of the loop is responsible for the gain?* An RSI loop contains ordinary on-policy RL as a special case; a positive result does not by itself show that recursion, the verifier, or the meta level mattered. Second, *is the gain real?* Benchmarks such as LIBERO [14] have a fixed set of official initial states, and it is tempting to train on them; we find that a widely used VLA-RL checkpoint trained this way is inflated by about 17 points relative to freshly sampled placements of the same tasks (Sec. 5.5).

We treat both questions as experimental-design problems. We build a framework (Sec. 3) in which each ingredient of RSI is an independent switch inside one engine: the verifier can be real or permuted, practice can be generated by the latest policy or by the frozen initial one, the meta level can be absent, a numeric search, or a large language model that reads the round records and edits the training procedure, and the whole budget can be spent in one round of plain RL. We then *preregister* every claim (Sec. 4): comparisons, bars, statistics and the read-once rule were committed before the runs, every change is a timestamped public amendment, the official test is sealed until all decisions are made, and finals are averaged over replicate evaluations.

Findings. Figure 1 summarises them.

1. **The gate makes self-training safe.** Over 31 runs with an OpenVLA-OFT policy on LIBERO-Goal, no gated run ends below its start beyond evaluation noise, while the same recipe collapses to 5.5% without the gate (Sec. 5.1).
2. **The loop's gains are small, and no single ingredient clears its bar.** The loop gains one to two points; recursion contributes $+1.7$ $[-0.3, +3.7]$ and the verifier $+1.2$ $[-0.4, +2.9]$; one round of plain RL at equal compute does as well (Sec. 5.2).
3. **A meta level does not help a procedure that already works, and we can show why.** Four variants, including an LLM that may edit nine harness components, are within ± 1.2 points of a fixed procedure; a second registered study on two platforms gives -0.3 and $+0.3$. The LLM's diagnoses are correct, but the object loop reaches the same plateau under every procedure in the space (Sec. 5.3).
4. **The meta level becomes distinguishable from RL only when the procedure fails.** Starting from a recipe that collapses without the gate, the LLM diagnoses the cause unprompted and its first proposal is kept by the gate on three of four seeds; the registered sealed test of this self-repair study is read on 2026-11-08 (Sec. 5.3).
5. **In a second domain the same loop gives larger, significant gains, and recursion pays only at long horizon.** Code world models with four LLMs: verified feedback $+2.6$ to $+5.8$ points; recursion $+2.8$ $[+1.2, +4.5]$ at eight rounds, invisible at four (Sec. 5.4).
6. **A published VLA-RL number owes about a fifth of itself to the test states** ($+16.8$ points $[+9.8, +23.4]$; Sec. 5.5).

Contributions. (i) An RSI framework for VLA policies in which every control is a switch inside one engine, with SFT, behaviour cloning and GRPO as interchangeable update operators and an LLM meta level that edits a declared harness space under an attribution guard. (ii) A preregistered, sealed-test protocol for self-improvement claims, with every amendment public. (iii) The first controlled decomposition of an RSI loop on a VLA benchmark, with its negative results and their mechanism. (iv) A positive recursion result in a second domain at longer horizon. (v) A measurement of train-on-test inflation in a published VLA-RL result. Code, preregistrations and all run records are released.

2. Related work

Self-improvement with verifiers. STaR [24], ReST [5] and ReST-EM [21] fine-tune a language model on its own verified rationales; SPIN [2] and self-rewarding models [23] close the loop with the model as its own judge; Absolute Zero [25] lets the model write its own tasks and uses a code executor as the verifier. RLVR with group-relative policy optimisation [3, 20] is the dominant update rule. These works report that the loop helps; our contribution is to ask which part of the loop helps, with every part switchable and preregistered. The closest analyses are the iterated on-policy convergence results of [12], whose predicted shape (early saturation of a learner restricted to samples from the initial policy) matches our code-domain recursion result.

Meta levels that edit the procedure. LLM-driven search over programs and training recipes has produced strong results when the search space is large and evaluation is cheap [6, 16, 19]. In our setting evaluation is expensive (hundreds of simulator episodes per candidate) and noisy, and the space is a few dozen knobs and harness components. We show that under these conditions an LLM meta level that diagnoses correctly still cannot beat a fixed procedure that already works, and we isolate the case where it could.

RL for VLA policies. OpenVLA [10] and OpenVLA-OFT [11] are the policies we improve; SimpleVLA-RL [13], RL4VLA [15] and related work [17] apply on-policy RL to such policies in simulation with a binary success reward. We use the SimpleVLA-RL training stack as one update operator and its released checkpoints as starting points. Our train-on-test measurement (Sec. 5.5) concerns the evaluation practice of this line of work rather than its methods.

Preregistration. Preregistered designs are standard in the experimental sciences [18] and have been proposed for machine learning [1]. We follow the practice literally: every bar, statistic and read-once rule was committed to a public repository before the corresponding runs, and every change is a dated amendment that states what had been seen.

3. Framework

3.1. Setting and notation

A policy π_θ acts in scenes $s \sim \mathcal{S}$ (object placements of a task, or programs with inputs); an attempt $\tau \sim \pi_\theta(\cdot \mid s)$ is a trajectory, and an external oracle $v(s, \tau) \in \{0, 1\}$ the policy cannot influence labels it (the simulator's success flag; a sandboxed interpreter). A *program* P is a JSON document with four sections—`curriculum` (which scenes to practise, how many, how allocated across tasks), `solver` (attempts per scene K, temperature T), `curation` (which attempts become training items) and `update` (the operator and its hyperparameters)—and $\mathcal{P}$ is a declared edit space of allowed values. A *version* is a pair (θ, P). The framework keeps an archive of versions; the one it currently trusts is the *retained* version $(\theta^\star, P^\star)$.

3.2. The loop (Algorithm 1)

One *inner step* runs program P from policy θ: propose scenes, attempt each K times, verify, curate, update. Curation `mixed_groups` keeps every attempt of scenes whose success rate lies strictly inside a band (used by GRPO [20]: the group-centred advantage is zero when all attempts agree); `verified` keeps successful attempts only (behaviour cloning, BC). The update is a LoRA step on θ. A

Algorithm 1 Gated recursive self-improvement with a meta level

Require: initial policy θ_0, seed program P_0, oracle v, gate set size n_g, rounds R, candidates per round c

$(\theta^\star, P^\star) \leftarrow (\theta_0, P_0)$; evaluate $\theta^\star$ on fresh gate and audit sets
for $r = 1 \dots R$ **do**
  draw a fresh gate set G_r of n_g paired placements; re-evaluate $\theta^\star$ on G_r
  $\{P_1, \dots, P_c\} \leftarrow \{P^\star\} \cup \textsc{Propose}(\text{round records}, \mathcal{P})$ ▷ meta level: fixed / numeric / LLM
  for each candidate program P_i **do**
    $\theta_i \leftarrow \theta^\star$
    for inner step $= 1 \dots 3$ **do**
      scenes $\leftarrow \textsc{Propose}(P_i, \theta_{\text{gen}})$ with $\theta_{\text{gen}} = \theta_i$ (recursive) or θ_0 (`nonrec`)
      attempts $\leftarrow K$ samples per scene at temperature T; labels $\leftarrow v(\cdot)$ (or permuted: `shuffled`)
      items $\leftarrow \textsc{Curate}$(attempts, P_i); $\theta_i \leftarrow \textsc{Update}(\theta_i, \text{items}, P_i)$
    end for
    $g_i \leftarrow$ success of θ_i on G_r (greedy); record audit, never use it
  end for
  $i^\star \leftarrow \arg\max_i g_i$; **if** $g_{i^\star} \geq g(\theta^\star, G_r)$ **then** $(\theta^\star, P^\star) \leftarrow (\theta_{i^\star}, P_{i^\star})$ ▷ gate
end for
read the sealed test of $\theta^\star$ once, after every run of the study has ended

round is three inner steps from the same parent, producing a *candidate* θ'. Because the next round starts from the retained version, practice is always generated by the latest policy: this is the *data recursion*, and the control `nonrec` breaks exactly it by proposing and attempting with the frozen initial policy θ_0 while still updating from $\theta^\star$.

3.3. Gate, audit and sealed test

Three disjoint evaluation sets are drawn per run from seeds that never overlap the official benchmark states. The *gate* (20 placements per task, 200 episodes, greedy decoding, environment re-seeded per episode so that versions are compared on identical placements) decides: a candidate is kept iff its gate success is at least the parent's on the same placements. In the *fresh-paired* mode used in every main run a new gate set is drawn every round and the parent is re-evaluated on it, so acceptance is always a paired comparison on placements never seen before, and a version cannot overfit a fixed gate. The *audit* (200 more) is recorded but never decides anything; it exposes gate overfitting. The *test* is the official benchmark (50 initial states per task, 500 episodes)

and is *sealed*: no process in the loop can read it, and it is opened once, after every decision in every run of a study has been made.

This is the *selection recursion*: the system's own verification result decides its next version, and the kept version decides who generates the next practice.

3.4. Meta level

The meta level versions the program and lets it be improved under the same gate. Each round the incumbent $P^\star$ and $c-1$ proposed programs are each trained for real from the same parent, and the gate picks the version to keep; a program is credited only when its child wins. Three proposers:

- `fixed`: no change (the meta level is off).
- `numeric`: random mutation of declared knobs, biased toward knobs whose past changes raised the gate.
- `llm`: a language model (Claude Sonnet) reads the current program, the edit space with a neutral description of what each option computes, and, for every past candidate, its program diff, gate score, acceptance and per-step statistics (sampled success, share of mixed scenes, item counts, first/last training loss). It must write a diagnosis, then return one new program.

The edit space has two sizes. The *knob* space has ten hyperparameters (scenes, allocation, samples, temperature, curation rule and budget, operator, learning rate, steps, batch). The *harness* space adds nine components of the procedure itself: the ratio computation of the policy gradient (legacy or matched), the loss aggregation (token or episode), the BC likelihood temperature (unit or rollout), advantage normalisation, the curation band, the per-scene cap, the allocation floor and the two clip bounds. Each corresponds to a change that was made by hand during development after diagnosing round records; the LLM is never told which option is better, so rediscovering them is a test of the meta level. This is the *procedure recursion*.

Attribution guard. The meta level never sees the audit or test, the policy weights, the scenes or the episodes; it only chooses values inside $\mathcal{P}$, every proposal passes the same validator as a numeric mutation, invalid proposals and failed calls fall back to numeric search and are counted, and a *compute cap* scales any proposal so that no candidate does more rollouts or optimiser work per step than the starting program. All practice data still comes from the policy's own attempts and all verdicts from the oracle; the LLM is a source of procedure choices, not of data.

3.5. Every control is a switch

Table 1 lists the arms used in the experiments. Each differs from `fixed` in exactly one line of Algorithm 1, so a difference between an arm and `fixed` is attributable to that line; `full` and the `llm` arms train the same number of candidates and so cost the same.

Table 1. **Controls.** Each arm flips one switch of Algorithm 1.

arm	change in Algorithm 1	isolates
`fixed`	none (proposer off)	gated recursive loop
`nonrec`	$\theta_{\text{gen}} = \theta_0$	data recursion
`shuffled`	labels permuted per scene	the verifier
`onebig`	R=1, $3R$ inner steps	gate + iteration vs one-shot RL
`full`	numeric proposer, c=2	meta level (equal compute)
`llm`∗	LLM proposer, c=2	meta level (equal compute)

∗ `llm`: knob space; `llm_harness`: harness space; `llm_harness_cap`: harness space under the compute cap.

4. Experimental setup

4.1. Benchmarks, policies and starting points

LIBERO / OpenVLA-OFT. LIBERO-Goal [14] has ten goal-conditioned tasks in one scene; LIBERO-10 ten long-horizon tasks. The policy is OpenVLA-OFT [11] (7B, action chunks of 8), started from a supervised fine-tune on *one* demonstration per task [13] so that there is room to improve (about 59% on LIBERO-Goal, 19% on LIBERO-10). Rollouts and GRPO/BC training use the SimpleVLA-RL stack with LoRA (r=32). The seed program—chosen from sealed pilots and frozen by amendment before any main run—is GRPO with learning rate 3×10^{-5}, matched-ratio log-probabilities, episode-balanced loss, 8 samples per scene at temperature 1.5, 160 scenes per inner step, 200 optimiser steps of batch 64; 6 rounds of 3 inner steps; fresh paired gates. **Code world models.** The model predicts the output of a Python program on an input (the CRUXEval-O format [4]); it writes its own programs (3-shot from a pool that starts with 16 hand-written seeds and grows with admitted programs; a program is admitted only if it is deterministic, literal-valued and not a near-duplicate of any held-out problem), the sandbox executes them, and an attempt is verified iff its answer equals the interpreter's. Qwen2.5-Coder 1.5B and 7B [7], Qwen3-8B [22] and Granite-3.3-8B [8] are trained with LoRA SFT on at most two verified traces per case; the gate is a 400-program validation set proposed by the *initial* model; tests are CRUXEval-O/I and LiveCodeBench-execution [9].

4.2. Arms and baselines

The LIBERO-Goal main study runs the eight arms of Sec. 3.5 with three seeds each (a fourth seed was added to the `nonrec`, `shuffled`, `onebig` and `fixed` arms by amendment before any of their runs ended), 31 runs in all, on two GPU platforms: a shared 8×A800 node and a 16-GPU cloud pool with its own evaluation stack. The stacks are not bit-identical, so every comparison is paired within a platform and the platforms serve as replicates. `onebig` is

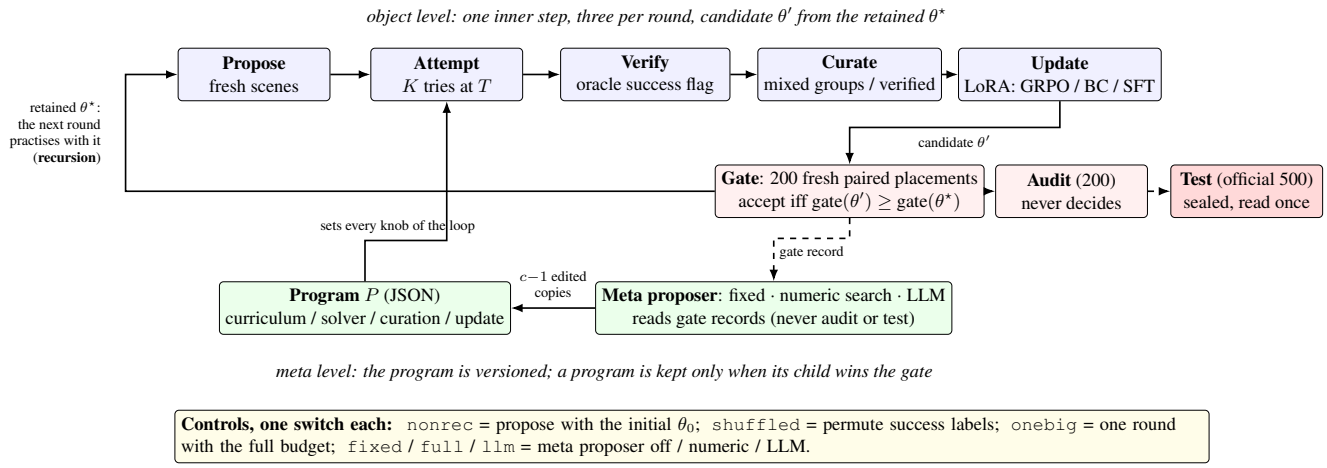


Figure 2. **The framework.** One engine, every ingredient a switch. The object loop practises on fresh scenes with the latest policy; the gate keeps a candidate only if it does not regress on paired held-out placements; the meta level versions the training program and may let a numeric search or an LLM edit it inside a declared space, seeing only gate records. Controls flip one switch each.

the plain-RL baseline: the whole six-round rollout and update budget in one round of 18 on-policy inner steps from the initial policy, with the same sampling and update code; it is read both through the gate and as its ungated single candidate. The code-domain study runs `fixed`, `shuffled`, `nonrec`, `full` and `onebig` for four rounds, plus an eight-round `fixed` vs `nonrec` study on two models.

4.3. Preregistered protocol

Self-improvement claims are easy to inflate: the loop selects versions, evaluation is noisy, and the experimenter sees intermediate numbers. We fixed the protocol before the runs and release the preregistration documents (26 dated amendments for the VLA study) with the code.

Sealing. The official LIBERO test states are never used for training, sampling, gating or program selection, and no test file is opened before a study's read-once. Only two sealed pilots per model were allowed before the seed program was frozen.

Finals and reference. A single 500-episode evaluation is itself noisy: the *same* initial policy evaluated in different runs scored between 57.2% and 60.0% (sd ≈ 0.7), and about 40% of episodes flip between nearby policies. Each run's final is therefore the per-episode mean of three evaluations of its retained policy (the run's own test and two replicate evaluations from the same plan and source tree), and the initial reference is the per-episode mean of every in-run evaluation of the initial policy on that platform (36 evaluations on the cloud pool, 21 on the A800). A noise check—the same statistic between two halves of the initial-policy evaluations—is reported with every verdict.

Claims and bars. Seed-averaged per-episode paired bootstrap (10,000 draws), 95% CI:

claim	comparison	bar
R1 improvement	`fixed`, `full` − reference	$\geq +5$, CI > 0,
R2 verifier	`fixed` − `shuffled`	$\geq +3$, CI > 0
R3 recursion	`fixed` − `nonrec`	$\geq +2$, CI > 0
R4 meta level	each meta arm − `fixed`	$\geq +2$, CI > 0
R5 vs one-shot RL	best gated arm (by gate) − `onebig`	$\geq +2$, CI > 0
M2-A meta v2	`llm` − identical-program replica	$\geq +3$, CI > 0
S1 self-repair	`llm_harness` − `fixed` (failing start)	$\geq +3$, CI > 0

Negative readings are reported as such; the reading scripts were committed before each read and their outputs are released.

5. Results

We organise the results around five questions. Every number below is from a preregistered read of a sealed test unless marked *gate-level*.

5.1. RQ1: Does the gated loop improve a VLA policy, and is it safe?

Finding 1: safe, and modestly effective. Figure 3 shows all 31 runs. Every gated run ends within noise of or above its start (finals 57.1–62.2% against a 58.4–58.9% reference; the two runs below their reference are within -1.3 and -1.7 points), the typical gain is one to two points, and the best single seed gains $+3.3$ $[+0.1, +6.6]$. The registered R1 claim (Tab. 2) is $+1.71$ $[-0.76, +4.23]$ for `fixed` and $+1.28$ for `full`, below the $+5$ bar. The safety is not a property of the recipe: in always-accept pilot curves on LIBERO-10 every recipe stayed within ± 3 points of its start

Table 2. **Registered claims on LIBERO-Goal** (official 500-episode test, 3-evaluation finals, pooled initial reference, seed-averaged paired bootstrap). No bar is met. Platform Y = cloud pool, A = A800.

claim	comparison	points	95% CI	bar	seeds
R1	`fixed` − ref. (Y)	+1.71	[−0.76, +4.23]	+5	1–3
R1	`full` − ref. (Y)	+1.28	[−1.37, +3.99]	+5	1–3
R2	`fixed` − `shuffled` (Y)	+1.22	[−0.37, +2.85]	+3	0–3
R3	`fixed` − `nonrec` (Y)	+1.72	[−0.28, +3.68]	+2	0–3
R4	`full` − `fixed` (Y)	−0.42	[−2.40, +1.56]	+2	1–3
R4	`llm_harness` − `fixed` (Y)	−0.73	[−2.58, +1.07]	+2	1–3
R4	`llm` − `fixed` (A)	+1.04	[−0.84, +3.00]	+2	1–3
R4	`llm_harness_cap` − `fixed` (A)	+1.16	[−0.76, +3.16]	+2	1–3
R5	best gated − `onebig` (Y)	−0.58	[−2.12, +0.95]	+2	0–3
noise	half vs. half of the n0 evaluations (Y / A)	+0.64 / +0.54	[−0.07, +1.36] / [−0.24, +1.35]		

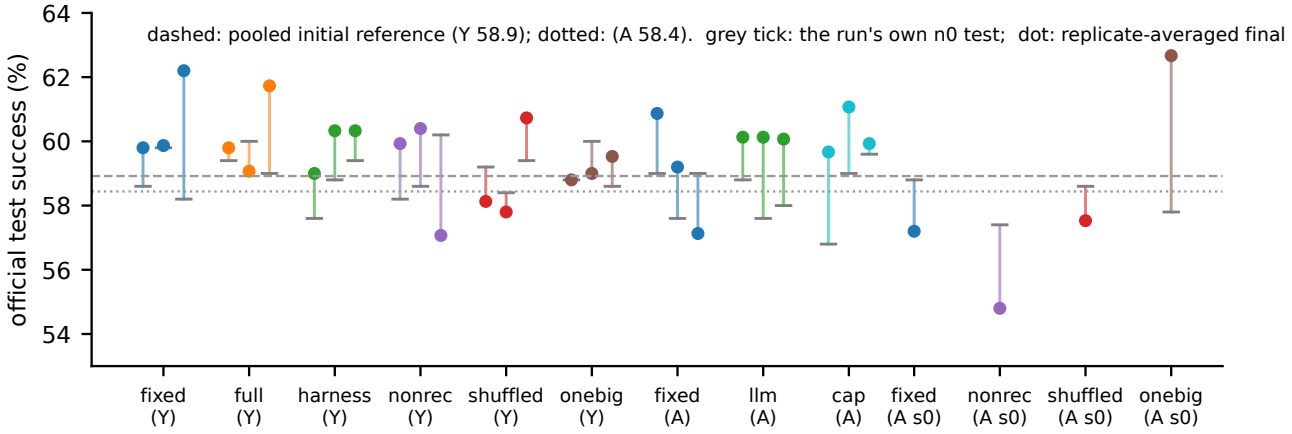


Figure 3. **Every run of the LIBERO-Goal study.** Grey tick: the run's own initial-policy test; dot: the replicate-averaged final of the retained policy; dashed/dotted lines: the pooled initial reference per platform. No gated run ends below its reference beyond noise; the one-shot `onebig` policy of seed 0 is the best single final.

for 18 on-policy steps, and the behaviour-cloning recipe of Sec. 5.3 collapsed from 58.5% to 5.5% when its versions were accepted unconditionally (Fig. 1a). Under fresh paired gates the same recipes never fall below their start on the sealed test. The price is that a 200-episode paired gate has a standard deviation of about 3.4 points, so it accepts and rejects many candidates on noise; acceptance rates are in the released records.

5.2. RQ2: Which part of the loop helps?

Finding 2: recursion and the verifier are the largest ingredients; neither clears its bar; plain RL ties. Recursion (R3) is +1.72 [−0.28, +3.68] against the +2 bar, the preliminary single-evaluation read had passed it at +2.40 and did not survive replicate averaging. The verifier (R2) is +1.22; without it the loop ends exactly at its start (57.8–60.7%). One round of plain RL with the same budget matches or beats the loop (R5 −0.58; on seed 0 the one-shot policy scores 62.7 against the gated loop's 57.2). Figure 4 shows the mechanism at the gate: `nonrec` stops climbing after round 3 while `fixed` continues, the same shape as in the code domain where the separation becomes significant at eight rounds (RQ4); a registered 12-round extension of R3 and R5 is running (amendment 27).

Table 3. **Per-arm finals** (%, mean of the 3-evaluation finals over seeds 1–3; seed-0 runs listed separately). Reference 58.9 (Y) / 58.4 (A).

	`fixed`	`full`	`llm`∗	`nonrec`	`shuf.`	`onebig`
Y, s1–3	60.6	60.2	59.9	59.1	58.9	59.1
A, s1–3	59.1	–	60.1 / 60.2	–	–	–
A, s0	57.2	–	–	54.8	57.5	62.7

∗ Y: `llm_harness`; A: `llm` / `llm_harness_cap`.

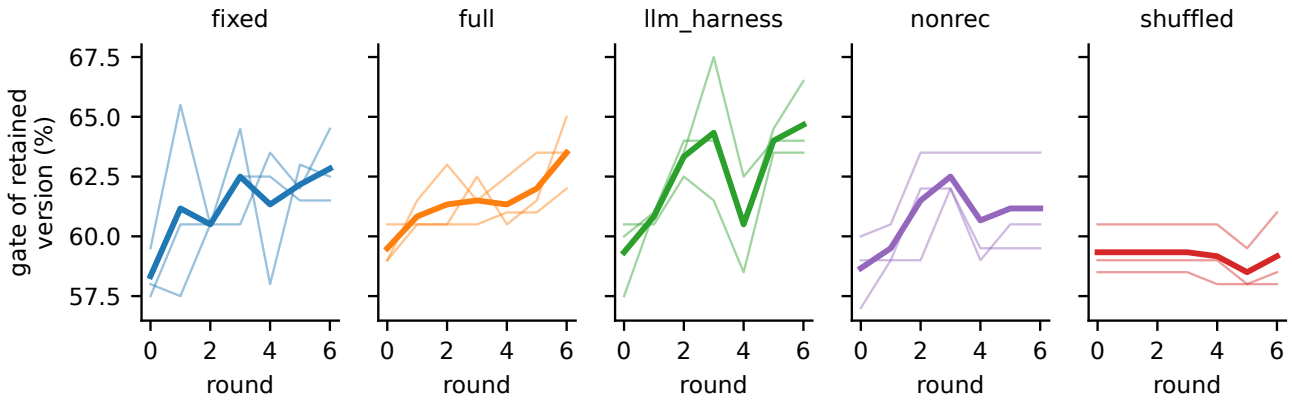


Figure 4. **Gate of the retained version by round** (cloud platform, seeds 1–3 thin, mean bold). The gated arms climb 3–5 points at the gate within six rounds; `shuffled` (verifier removed) stays flat; `nonrec` (practice from the frozen initial policy) stops after round 3. Gate gains overstate test gains by the winner's curse (Tab. 3).

5.3. RQ3: Does a meta level help, and when could it?

Finding 3: no meta level beats a working fixed procedure, on two studies and two platforms. All four R4 arms are within ± 1.2 points of `fixed` (Tab. 2). Because those arms compare only each round's newest edit against the incumbent and let the LLM see no per-task signal, we registered a second study ("v2") in which the LLM sees per-task gate rates with their noise, each candidate starts from its parent's statistics, and the comparison is between an `llm` arm and an identical-program *replica* arm at the same compute, from two starting policies. The primary bar fails on both platforms: -0.27 $[-2.53, +2.00]$ (cloud, seeds 1–3) and $+0.30$ $[-1.60, +2.15]$ (A800, seeds 0–3); pooled over seven seed-pairs any true effect is within about ± 2 points.

Finding 4: the null has a mechanism. Over the v2 study the LLM's proposals scored -1.2 points against the incumbent at the gate (n=30) while identical-program replicas scored $+0.6$ (sd 3.4, n=36): there is no edit effect for a finer gate to resolve. The LLM's diagnoses, logged verbatim, read the loop correctly—the task that is never solved (so GRPO has no signal), the saturated task, the remaining gate changes within noise—and its edits are the ones a practitioner would try. None moves success within six rounds: final retained gates are 62.8 (`fixed`), 63.5 (`full`), 65.0 (`llm_harness`) and 61.7 (`onebig`), all within gate noise. *The object loop reaches the same plateau under every procedure in the space*, so a correct diagnosis of a procedure that already works has nothing to repair.

Self-repair: a procedure that fails. A meta level is distinguishable from RL only where RL's procedure fails and a different procedure would not. The study contains one such case, found before any meta-level run: BC on the policy's own temperature-1.6 samples with a unit-temperature likelihood collapses when its versions are accepted unconditionally (58.5% $\rightarrow$ 5.5%). The repair is a one-knob change (the BC temperature, the sampling temperature, or the operator), and all three are in the harness space. We registered a study (amendments 24 and 26) before any of its runs: start from the collapsing recipe; arms `fixed` (the gate alone), `full` (numeric search) and `llm_harness` (same compute); four seeds; bars S1 = `llm_harness` $-$ `fixed` $\geq +3$ with CI above 0, S2 = `llm_harness` $-$ `full` reported with its CI. The failure case was written down in advance: if S1 fails, the meta level has no demonstrated effect in this study.

Finding 5 (gate-level, not a claim): the gate alone prevents the collapse, and the LLM diagnoses the failure unprompted. Figure 5 and Tab. 4 give the record. No `fixed` run falls below its start; one seed is held at the initial policy for all six rounds, the other three accept small BC gains (60–64.5%). The LLM's first-round diagnoses on all four seeds name the sampling/likelihood temperature mismatch and the missing failure signal (verbatim in the released logs), and its first proposal on every seed is the same—switch to GRPO on mixed-outcome scenes at a lower temperature; the gate kept that switch on three seeds (gates 64.5–65.5%) and refused it twice on the fourth, where the LLM then repaired BC instead (lower sampling temperature; retained gate 64.5%). Numeric search also finds repairs (a GRPO switch on one seed, the temperature-matched BC likelihood on another). At the gate all arms end within noise of each other (60–65.5%), so S1 and S2 are decided by the sealed test, read on 2026-11-08. A second failing scenario was sought with six gate-only pilots (four extreme GRPO/BC recipes, both platforms); none degraded under the gate, so no second scenario was registered.

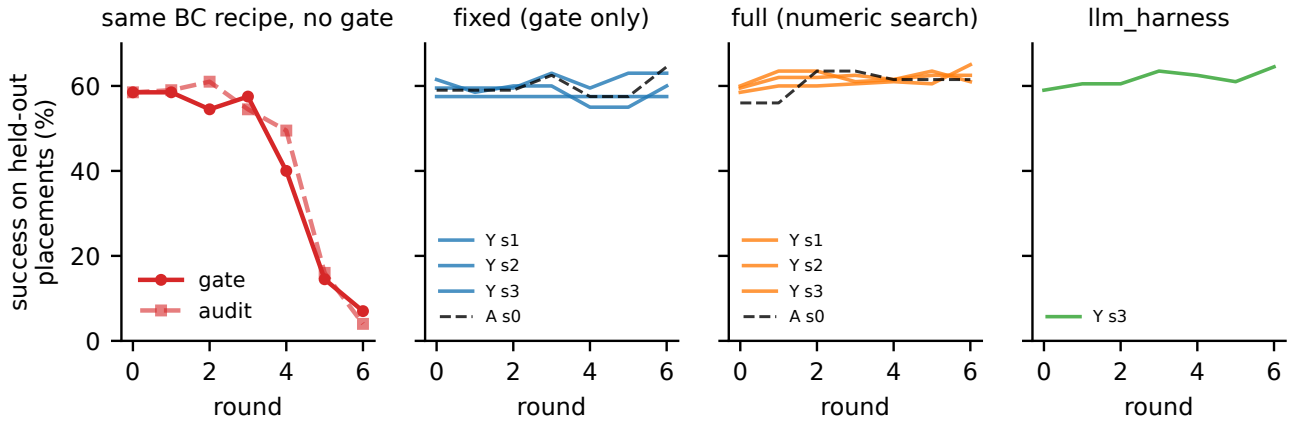


Figure 5. **Self-repair study, gate level.** Left: the starting BC recipe collapses when every version is accepted. Right: under the gate the same recipe never falls below its start (`fixed`); both proposers find repairs that reach 64–65% at the gate; the arms are within gate noise of each other, so the sealed test decides S1 and S2. Runs still in progress are absent.

Table 4. **Self-repair study, gate-level record** (retained version and its gate, %; what the meta level did). Sealed test results: [to be inserted after the registered read of 2026-11-08].

seed	`fixed` (gate only)	`full` (numeric search)	`llm_harness`
1 (Y)	r6, 60.0; BC unchanged	r5, 62.5; BC unchanged	*running*; LLM switched to GRPO in round 1, kept (65.5 by r3)
2 (Y)	n0 (all 6 refused)	r6, 65.0; switched to GRPO	*running*; LLM switched to GRPO in round 1, kept (64.5, 64.0, 65.5)
3 (Y)	r5, 63.0; BC unchanged	r6, 61.0; BC, rollout-temp. likelihood	r6, 64.5; GRPO refused twice, BC repaired: T 1.6→1.3
0 (A)	r6, 64.5; BC unchanged	r4, 64.0 (rounds 5–6 refused)	*running*; LLM's GRPO child kept in round 2 (65.5 vs 62.5)

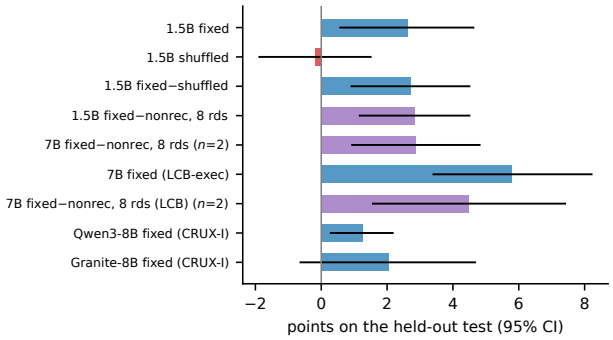


Figure 6. **Code world models.** Verified self-training gains (blue), the permuted-label control (red) and the recursion contrast at eight rounds (purple), with 95% CIs.

5.4. RQ4: Does the picture transfer to a second domain?

Finding 6: verified feedback gives significant gains; recursion separates only after the frozen generator is used up. The verified loop improves every model on at least one test (Tab. 5, Fig. 6: CRUXEval-O +2.6 for the 1.5B model, LCB-execution +5.8 for the 7B model, CIs excluding 0), the permuted-label control does not, and the difference is significant on the trained task family (+2.7 $[0.9, 4.5]$). The numeric meta level does not beat the fixed program in any three-seed cell. Recursion is not detectable at four rounds (-0.6) but is significant at eight (+2.8 $[1.2, 4.5]$, replicated on the 7B model at +2.9 $[0.9, 4.8]$ and +4.5 $[1.6, 7.4]$ on LCB-execution): the frozen generator's practice is as good as the improving model's for about four rounds, after which the non-recursive arm stops accepting gate gains (2 vs 8 accepted gains in rounds 5–8) while the recursive arm keeps finding cases at its new frontier. The extra gain is on the trained task family with no extra forgetting on the secondary suites—the shape predicted for iterated on-policy learning [12], and the reason we expect the VLA recursion result (+1.7 at six rounds) to be on the same curve before it separates.

5.5. RQ5: How much of a published VLA-RL number is the test set?

Finding 7: about a fifth of the headline number. LIBERO is tested on 50 fixed initial states per task, and the released SimpleVLA-RL LIBERO-10 checkpoint was trained by RL on exactly those states. We evaluated it and its own SFT starting point on the 500 official episodes and on 500 placements freshly drawn by LIBERO's own sampler (the

Table 5. **Code world models**, final − initial on the held-out tests (points, seed-averaged paired bootstrap, 95% CI). Four rounds unless noted; "8 rds" = the long-horizon recursion study; † two seeds.

model	comparison	CRUXEval-O	CRUXEval-I	LCB-exec
Qwen2.5-C 1.5B	`fixed` − init	$+2.6\ [0.6, 4.6]$	$+1.2\ [-0.8, 3.3]$	$-1.3\ [-4.4, 1.7]$
	`shuffled` − init	$-0.2\ [-1.9, 1.5]$	$-0.1\ [-1.5, 1.3]$	$-2.4\ [-4.7, 0.0]$
	`fixed` − `shuffled`	$+2.7\ [0.9, 4.5]$	$+1.3\ [-0.5, 3.0]$	$+0.9\ [-1.6, 3.5]$
	`full` − `fixed`	$+0.8\ [-0.7, 2.4]$	$-0.8\ [-2.3, 0.6]$	$+0.7\ [-1.7, 3.1]$
	`fixed` − `nonrec`	$-0.6\ [-2.2, 1.1]$	$+0.3\ [-1.3, 1.8]$	$+0.9\ [-1.3, 3.1]$
	`fixed` − `nonrec`, 8 rds	$+2.8\ [1.2, 4.5]$	$+0.2\ [-1.5, 1.9]$	$-0.5\ [-3.1, 2.1]$
Qwen2.5-C 7B	`fixed` − init	$+1.4\ [-0.2, 3.0]$	$+2.0\ [0.2, 3.8]$	$+5.8\ [3.4, 8.2]$
	`full` − `fixed`	$+0.8\ [-0.5, 2.0]$	$+0.1\ [-1.5, 1.6]$	$+1.1\ [-1.0, 3.1]$
	`fixed` − `nonrec`, 8 rds†	$+2.9\ [0.9, 4.8]$	$0.0\ [-2.1, 2.1]$	$+4.5\ [1.6, 7.4]$
Qwen3-8B	`fixed` − init	$+0.5\ [-0.2, 1.3]$	$+1.3\ [0.3, 2.2]$	$+0.2\ [-0.5, 0.9]$
	`full` − `fixed`†	$+0.7\ [-0.8, 2.2]$	$+2.6\ [0.6, 4.7]$	$+0.5\ [-1.0, 2.1]$
Granite-3.3 8B	`fixed` − init	$+1.4\ [-0.8, 3.7]$	$+2.0\ [-0.6, 4.7]$	$+2.4\ [-1.0, 5.9]$

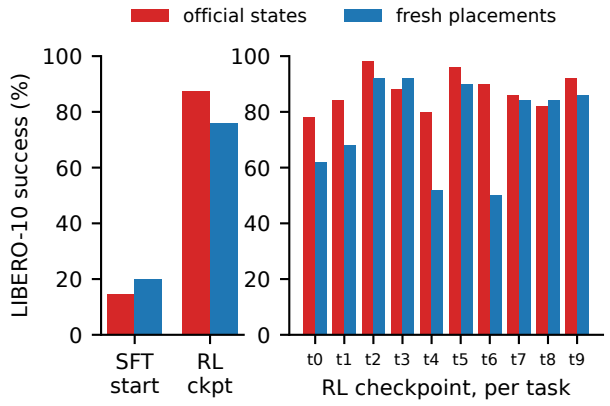


Figure 7. **Train-on-test inflation** (LIBERO-10). Left: official vs. fresh placements for the SFT start and the RL checkpoint trained on the official states. Right: the RL checkpoint per task; the loss is concentrated on tasks 4 and 6.

same distribution, not a perturbation): SFT 14.6% official / 20.0% fresh; RL 87.4% / 76.0%. The difference of the two gaps, which removes any difficulty difference between the placement sets, is $+16.8$ points $[+9.8, +23.4]$; a replication on 500 further never-seen placements gives $+16.0$ $[+9.2, +22.4]$ (Fig. 7). The checkpoint is still far better than its start on fresh placements ($+56$ points), so the method works; the point is that about a fifth of the headline number is memorisation of the test states. Every number in this paper is measured on the official states *without* ever training, sampling or selecting on them, so our finals should be compared with the "fresh" column of such results.

6. Discussion

What we claim. (1) A gated self-improvement loop for VLA policies is safe at this budget: with fresh paired gates, no run degrades on the sealed test, including runs whose recipe collapses without the gate. (2) Its gains on LIBERO-Goal are small, one to two points, and are not distinguishable from one round of plain RL at equal compute; recursion is the largest ingredient and sits at the edge of its bar. (3) A meta level that edits hyperparameters or harness components does not beat a fixed, tuned procedure, on two studies and two platforms, and the reason is mechanistic: every procedure in the space reaches the same plateau within six rounds. (4) In the code domain the verified loop gains several points and recursion is significant at eight rounds. (5) A published VLA-RL number is inflated by about 17 points by training on the test states.

What we do not claim. We do not claim that recursion or the verifier are useless for VLA policies: the point estimates are positive on every comparison, and the code-domain result shows that recursion separates only after the frozen generator's practice is used up, which six rounds on LIBERO-Goal may not reach; a registered 12-round study is running. We do not claim that an LLM meta level cannot help: the self-repair study (sealed until 2026-11-08) is the test that can show it, and the paper reports its registered result either way.

For practitioners. Three things in these records are directly usable. A fresh paired gate turns an unsafe self-training recipe into a safe one at the cost of about one extra evaluation per candidate, and it is the only ingredient whose effect is unambiguous here. Replicate-averaged finals and a pooled initial reference are necessary at this scale: a single

500-episode evaluation carries ± 1.5 points of noise, enough to turn a preliminary pass into a registered fail, as happened to our R3. And an LLM meta level is worth its cost only where the procedure can fail; on a tuned recipe its correct diagnoses buy nothing.

Limitations. The VLA results are on one simulated benchmark family with a 7B policy and a few-demonstration start; longer horizons, other policies and real robots are untested here (real-robot data collection is in progress). The gate is noisy (sd 3.4 points at 200 episodes), which both limits the meta level and makes the loop conservative; a larger gate costs proportionally more simulation. Evaluation noise on the official test is about ± 1.5 points per evaluation, which is why finals are replicate-averaged and why bars below $+2$ would not be readable at this budget. Some runs were continued after hardware or wall-time interruptions, in each case from a completed round with the same plan; the amendments disclose every such event. Finally, preregistration constrains the analysis but not the design: the seed program and the harness space were chosen by us, and a different space might leave more for a meta level to find.

7. Conclusion

We built a recursive self-improvement framework in which every ingredient is a switch, preregistered every claim with a sealed test, and asked which part of the loop improves a VLA policy. The gate makes the loop safe; recursion and the verifier each contribute about a point and a half; a meta level that edits the procedure contributes nothing once the procedure works, for a reason the records show; and in a second domain recursion pays only beyond four rounds. We also found that a published VLA-RL number owes about a fifth of itself to the benchmark's fixed test states. We hope the protocol—sealed tests, fresh placements, replicate-averaged finals, public amendments, read-once—is as useful as the results, and that the self-repair study settles, one way or the other, when a meta level is worth its cost.

References

[1] Samuel Albanie, James Thewlis, Sébastien Ehrhardt, and João F. Henriques. On the preregistration of machine learning experiments. *NeurIPS Workshop on Preregistration in ML*, 2021. 3

[2] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. In *ICML*, 2024. 2

[3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025. 2

[4] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *ICML*, 2024. 4

[5] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. *arXiv:2308.08998*, 2023. 2

[6] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*, 2025. 3

[7] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2.5-Coder technical report. *arXiv:2409.12186*, 2024. 4

[8] IBM Granite Team. Granite 3.x language models. `https://github.com/ibm-granite/granite-3.0-language-models`, 2024. 4

[9] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv:2403.07974*, 2024. 4

[10] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *CoRL*, 2024. 3

[11] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv:2502.19645*, 2025. 3, 4

[12] Kim et al. Coverage improvement and fast convergence of on-policy preference learning. *arXiv:2601.08421*, 2026. 2, 8

[13] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaoyang Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. *arXiv:2509.09674*, 2025. 2, 3, 4

[14] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023. 2, 4

[15] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can RL bring to VLA generalization? an empirical study. *arXiv:2505.19789*, 2025. 2, 3

[16] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024. 3

[17] Guanxing Lu, Wenkai Guo, Chaoyi Zhang, Yue Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv:2505.18719*, 2025. 3

[18] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *PNAS*, 115(11):2600–2606, 2018. 3

[19] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*, 2025. 3

[20] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024. 2, 3

[21] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. In *TMLR*, 2024. 2

[22] An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025. 4

[23] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In *ICML*, 2024. 2

[24] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *NeurIPS*, 2022. 2

[25] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv:2505.03335*, 2025. 2